



PostgreSQL

для администраторов баз данных
и разработчиков

Меня хорошо видно & слышно?



Защита проекта

Тема: Миграция данных и хранимых процедур с Firebird 3.0 на PostgreSQL 16



Бутыгин Владимир

Программист-аналитик

ООО «РОЛЬФ Тех»

ООО «АСС-Бизнес софт»



План защиты



Цели проекта и предыстория

Что планировалось

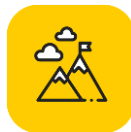
Используемые инструменты

Что получилось

Схемы/архитектура

Выводы

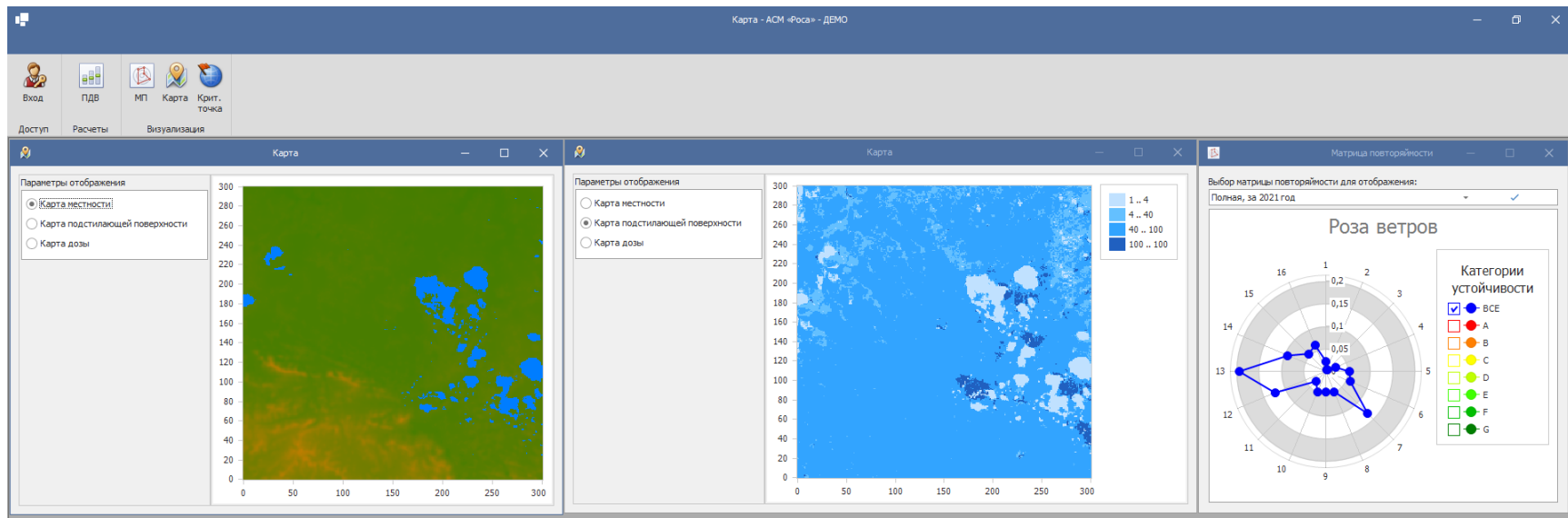
Цели проекта



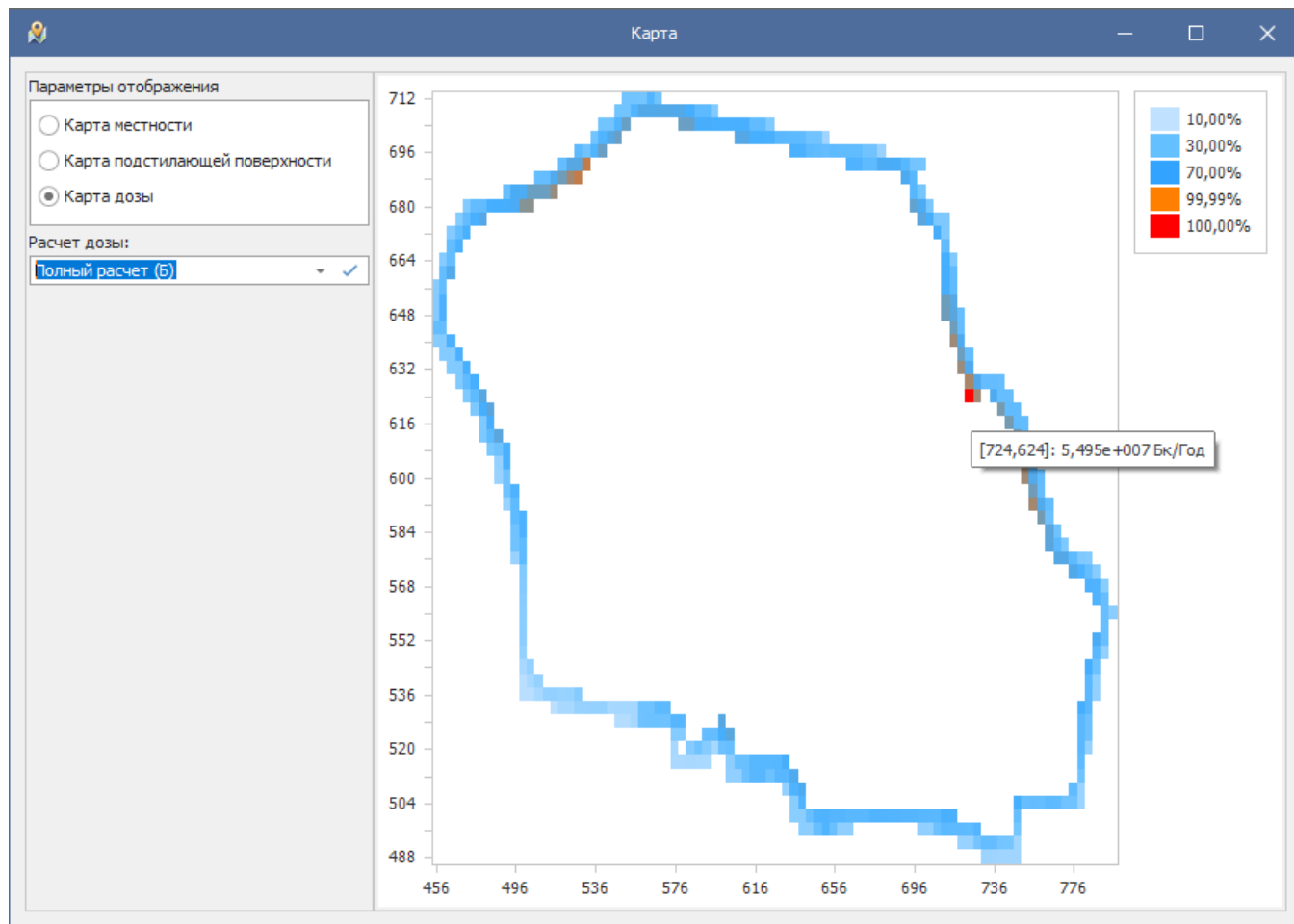
1. Оценка возможностей миграции с Firebird на PostgreSQL
2. Повышение производительности или точности вычислений



Предыстория



На VIII Международной молодежной научной конференции Физика. Технологии. Инновации ФТИ-2021, был предоставлен доклад на тему систематизации и автоматизации расчетов ПДВ.



Что планировалось



1. Перенос структуры хранения данных
2. Перенос хранимых функций и процедур
3. Оценка результатов вычислений



Используемые инструменты



1. СУБД Firebird 3.0 (Windows 10)
2. IBExpert (Windows 10)
3. СУБД PostgreSQL 16 (WSL: Ubuntu 24.04)
4. DBeaver CE (Windows 10)



Что получилось



1. Увеличить время вычислений за счет смены СУБД не удалось

Firebird: **1 час 30 мин.**

PostgreSQL: **1 час 50 мин.**

2. Точность вычислений выросла.

Есть предположение что увеличение времени вызвано тем что Firebird работал на хост машине, а PG под WSL, а также за счет повышения точности.

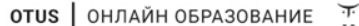
Мысли: интрузивные

Действия: глупенькие



Результаты: сомнительные

Шалость: удалась



Код обработки вычислений



```
1 do
2 $$
3 declare
4   p_rec record;
5   s_rec record;
6   l_counter int = 1;
7 begin
8   raise notice 'Start: %', now();
9   for p_rec in (select id from point p where p.id_area in (3)) -- 28612 rec
10  loop
11     for s_rec in (select id from "source" s) -- 10 rec
12    loop
13       call calc_dilutionfactor(1::bigint, 16::bigint, s_rec.id, p_rec.id, 0::bigint, 1::smallint, 0::smallint, 0::smallint);
14       l_counter = l_counter + 1;
15       commit;
16    end loop;
17     raise notice 'Point: %, Cnt: %', p_rec.id, l_counter;
18  end loop;
19  raise notice 'End: %. Result size: %', now(), l_counter;
20  commit;
21 end;
22 $$
```



```

3 HashAggregate (cost=106.73..107.16 rows=9 width=320) (actual time=25.675..25.781 rows=10 loops=1)
4   Group Key: c.p_id, c.x, sn.id_nuclide
5   Batches: 1 Memory Usage: 272kB
6   CTE pointdata
7     -> Nested Loop Left Join (cost=1.42..33.79 rows=1 width=234) (actual time=7.749..7.758 rows=1 loops=1)
8       -> Nested Loop Left Join (cost=0.99..25.07 rows=1 width=198) (actual time=5.478..5.486 rows=1 loops=1)
9         -> Nested Loop Left Join (cost=0.85..16.90 rows=1 width=16) (actual time=5.052..5.056 rows=1 loops=1)
10           -> Index Scan using point_pkey on point p (cost=0.43..8.45 rows=1 width=16) (actual time=3.172..3.173 rows=1 loops=1)
11             Index Cond: (id = 676748)
12           -> Index Only Scan using pointdescription_id_point_idx on pointdescription pd (cost=0.43..8.45 rows=1 width=8) (actual time=1.865..1.866 rows=1 loops=1)
13             Index Cond: (id_point = 676748)
14             Heap Fetches: 0
15         -> Index Scan using source_pkey on source s (cost=0.14..8.16 rows=1 width=182) (actual time=0.416..0.417 rows=1 loops=1)
16           Index Cond: (id = 16)
17       -> Index Scan using point_pkey on point p2 (cost=0.43..8.45 rows=1 width=16) (actual time=0.541..0.541 rows=1 loops=1)
18         Index Cond: (id = s.id_point)
19 CTE caldata
20   -> Hash Right Join (cost=43.66..63.47 rows=1 width=176) (actual time=13.566..15.297 rows=13 loops=1)
21     Hash Cond: ((f.z0 = p.l.z) AND (r.wind_direction = (p.l.r)::numeric))
22   -> Hash Join (cost=43.63..62.30 rows=98 width=448) (actual time=3.841..4.159 rows=1506 loops=1)
23     Hash Cond: (f.z0 = e.hru_surface)
24     -> Seq Scan on spr_fx_coef f (cost=0.00..14.10 rows=410 width=160) (actual time=0.672..0.676 rows=6 loops=1)
25     -> Hash (cost=43.03..43.03 rows=48 width=320) (actual time=3.053..3.058 rows=1506 loops=1)
26       Buckets: 2048 (originally 1024) Batches: 1 (originally 1) Memory Usage: 146kB
27       -> Hash Join (cost=23.20..43.03 rows=48 width=320) (actual time=2.465..2.686 rows=1506 loops=1)
28         Hash Cond: ((e.sustainabilitycategory)::text = (g.sustainabilitycategory)::text)
29         -> Seq Scan on spr_epsilon e (cost=0.00..16.80 rows=680 width=84) (actual time=0.874..0.877 rows=42 loops=1)
30         -> Hash (cost=23.02..23.02 rows=14 width=298) (actual time=1.569..1.572 rows=251 loops=1)
31           Buckets: 1024 Batches: 1 Memory Usage: 29kB
32           -> Hash Join (cost=15.80..23.02 rows=14 width=298) (actual time=1.085..1.499 rows=251 loops=1)
33             Hash Cond: ((r.stability_type)::text = (g.sustainabilitycategory)::text)
34             -> Seq Scan on rmatrixdata r (cost=0.00..6.14 rows=251 width=98) (actual time=0.349..0.684 rows=251 loops=1)
35               Filter: (id_rmatrix = 16)
36             -> Hash (cost=15.67..15.67 rows=11 width=200) (actual time=0.709..0.710 rows=7 loops=1)
37               Buckets: 1024 Batches: 1 Memory Usage: 9kB
38               -> Hash Join (cost=1.16..15.67 rows=11 width=200) (actual time=0.684..0.689 rows=7 loops=1)
39                 Hash Cond: ((g.sustainabilitycategory)::text = (sz.sustainabilitycategory)::text)
40                 -> Seq Scan on spr_gx_coef g (cost=0.00..13.20 rows=320 width=148) (actual time=0.300..0.301 rows=7 loops=1)
41                 -> Hash (cost=1.07..1.07 rows=7 width=52) (actual time=0.357..0.357 rows=7 loops=1)
42                   Buckets: 1024 Batches: 1 Memory Usage: 9kB
43                   -> Seq Scan on spr_sigmar_max sz (cost=0.00..1.07 rows=7 width=52) (actual time=0.337..0.340 rows=7 loops=1)
44   -> Hash (cost=0.02..0.02 rows=1 width=84) (actual time=7.768..7.768 rows=1 loops=1)
45     Buckets: 1024 Batches: 1 Memory Usage: 9kB
46   -> CTE Scan on pointdata p_l (cost=0.00..0.02 rows=1 width=84) (actual time=7.755..7.762 rows=1 loops=1)
47 -> Nested Loop Left Join (cost=3.12..7.02 rows=10 width=376) (actual time=16.158..18.353 rows=130 loops=1)
48   -> CTE Scan on caldata c (cost=0.00..0.02 rows=1 width=176) (actual time=13.573..15.329 rows=13 loops=1)
49   -> Hash Left Join (cost=3.12..6.90 rows=10 width=200) (actual time=0.205..0.226 rows=10 loops=13)
50     Hash Cond: (sn.id_nuclide = np.id_nuclide)
51     -> Seq Scan on source_nuclide sn (cost=0.00..3.65 rows=10 width=72) (actual time=0.197..0.212 rows=10 loops=13)
52       Filter: (id_source = 16)
53       Rows Removed by Filter: 122
54     -> Hash (cost=1.94..1.94 rows=94 width=136) (actual time=0.064..0.064 rows=94 loops=1)
55       Buckets: 1024 Batches: 1 Memory Usage: 14kB
56       -> Seq Scan on nuclideparticle np (cost=0.00..1.94 rows=94 width=136) (actual time=0.013..0.027 rows=94 loops=1)
57
58 Planning Time: 24.721 ms
59 Execution Time: 31.412 ms

```

Plan. time:

24.722 ms

Exec. time:

31.412 ms

Выводы и планы по развитию



1. Очень близкие по стандарту между собой диалекты SQL
2. PG поддерживает гораздо больше число знаков после запятой
3. Возможно альтернативные способы вычисления могут дать меньшее время
4. Текущая реализация все также проигрывает многопоточным вычислениям реализованным на ЯП (например Java)



Спасибо за внимание!